Project Report
Automated Logistics and Access Security Gate with Monitoring and Behavioral Analysis

ISTE730 - Foundations of IoT
Student: Vetrivel Maheswaran
Instructor: Tae Oh

Abstract:

Affordable access control systems in industrial and logistics environments have traditionally focused on authentication alone, with no capability for behavioral monitoring, event logging, or intelligent anomaly detection. This project designs and implements an Automated Logistics and Access Security Gate system that addresses this gap by combining RFID-based authentication with real-time monitoring, behavioral analysis, and hybrid anomaly detection - all running locally on commodity hardware without proprietary infrastructure or cloud dependency. The system was built on an embedded microcontroller platform connected via serial communication to a Python backend. Each access event is validated, stored in a local database, and analyzed through a two-layer anomaly detection pipeline combining rule-based checks and an Isolation Forest unsupervised machine learning model. A live web dashboard provides real-time visibility into access activity, user presence, and behavioral analytics. The system successfully authenticated users, logged all access events with timestamps, tracked per-user entry and exit states, and detected anomalies across all implemented detection paths including off-hours access, rapid scanning, consecutive denied attempts, and statistically unusual behavioral patterns identified by the machine learning layer. End-to-end latency from card scan to dashboard update averaged one to two seconds. This work demonstrates that intelligent access monitoring - real-time logging, behavioral analysis, and machine learning anomaly detection - is achievable on a platform costing under sixty dollars, making it a practical alternative for environments where enterprise solutions are inaccessible.

Keywords: RFID, IoT, Access Control, Anomaly Detection, Isolation Forest, Arduino, FastAPI, Behavioral Analysis

1. Introduction

Access control has been a fundamental requirement in industrial and commercial environments for decades. From simple lock-and-key mechanisms to proximity card readers, the evolution of entry systems has been driven by the need for better security, accountability, and operational efficiency. However, most commercially deployed systems today still function in isolation - they authenticate a user at the door and nothing more. There is no persistent record of who entered, when they left, how long they stayed, or whether their behavior was consistent with their normal patterns. This gap between hardware-level authentication and data-driven intelligence is addressed in this project by developing an IoT-based access control system that not only authenticates users using RFID but also logs access events, enables real-time monitoring through a web dashboard, and applies basic behavioral analysis to detect unusual patterns.

The system integrates edge-level embedded hardware with a Python backend and a live web dashboard, creating a complete end-to-end IoT pipeline from physical card scan to real-time data visualization and behavioral analysis. The hardware and software design decisions are described in detail in Section 3. The sections that follow cover the background of RFID-based access control and IoT security systems, the hardware and software design of this prototype, the implementation of the detection pipeline, challenges encountered during development, and conclusions drawn from the project. The goal throughout is not just to describe a working gate but to explain the engineering decisions made at every layer of the system. This topic was selected because it sits at the intersection of embedded systems, backend software engineering, and data-driven security - three areas that are individually well studied but are rarely integrated at this scale in a single working prototype. The motivation, problem definition, and target population are discussed in Section 1.1.

1.1 Problem Statement

Modern access control systems deployed in warehouses, logistics centers, and industrial facilities are predominantly authentication-only solutions. Once a credential is verified and a gate opens or closes, the

system provides no further value. There is no persistent audit trail, no tracking of user presence, no detection of repeated unauthorized attempts, and no behavioral pattern analysis. This creates a measurable security gap. An Orange Labs survey found that 45% of supply chain executives rely on real-time data insights for operational decisions, yet most gate systems generate none of that data. Enterprise platforms from vendors such as EAIGLE, Opendock, and ASSA ABLOY address this effectively at scale, but depend on proprietary hardware, cloud infrastructure, and significant capital investment that places them beyond the reach of most small and mid-sized deployments. This project addresses that gap by building a fully local, open-source access control system on commodity hardware that delivers the same core capabilities - event logging, real-time monitoring, and intelligent behavioral anomaly detection without proprietary dependency or enterprise budget.

2. Background

2.1 RFID Technology in Access Control

Radio Frequency Identification, more commonly known as RFID is a wireless communication system where electromagnetic fields are used to locate and monitor tags placed on objects or on persons carrying the tags. The High Frequency (HF) range of 13.56 MHz is the most popular frequency band in access control applications, and is specified by the ISO 14443 standard. Cards with this frequency like MIFARE Classic cards produced by NXP Semiconductors can be read within a range of 10 centimeters and can be used in controlled entry applications where the user must be within close range of the reader [10].

RC522 is a very closely integrated reader IC manufactured by NXP that can be used in the ISO 14443A standard and communication with a host microcontroller through the Serial Peripheral Interface (SPI) protocol. SPI is a fast (4 wires) synchronous, full-duplex, communication standard that uses a single master and one or multiple slave devices. On the Arduino UNO board, the SPI hardware is hardwired to digital pins 11,12 and 13 respectively to MOSI, MISO and SCK, and a chip select line is configurable. This hardware constraint directly influenced the pin mapping decisions in this project.

According to Weinstein (2005), RFID deployment in supply chain and logistics environments has demonstrated measurable improvements in inventory accuracy and access accountability compared to barcode-based systems. The key advantage is that RFID does not require line-of-sight between the reader and the tag, and multiple tags can theoretically be read simultaneously, though in single-reader access control scenarios the system reads one card at a time [12]. The limitation of RFID in security applications is that the cards themselves can be cloned using off-the-shelf hardware, which is why this project treats the RFID layer as a first factor of authentication rather than the sole security mechanism.

2.2 IoT-Based Security Systems

The Internet of Things represents a paradigm in which physical devices are connected to computational infrastructure for the purpose of data collection, remote monitoring, and automated response. In the security domain, IoT-based systems have expanded beyond simple alarm triggers to encompass continuous behavioral monitoring [1]. Atzori, Iera, and Morabito (2010) characterize IoT as a network of connected objects that can sense, communicate and respond to their surroundings, and identify security as one of the main application verticals that lead to the adoption of the IoT.

Commercial access control systems typically couple RFID authentication with backend software for logging and reporting [13]. Enterprise implementations may include biometric secondary factors, network-connected readers, and centralized identity management systems. While reliable and scalable, these platforms are costly and demand significant infrastructure, creating demand for lightweight open-

source alternatives deployable on commodity hardware. A detailed review of commercial solutions is provided in Section 2.4.

2.3 Anomaly Detection in Security Contexts

The application of machine learning to detect anomalies in access patterns is an active research area [14]. Liu, Ting, and Zhou (2008) introduced the Isolation Forest algorithm, which isolates anomalies by randomly partitioning data space. Unlike density-based methods such as Local Outlier Factor, Isolation Forest does not require computing distances between all data points, making it computationally efficient for real-time applications on modest hardware. The algorithm assigns an anomaly score to each data point based on the average path length required to isolate it - anomalies are isolated in fewer partitions than normal points.

In the context of access control, behavioral anomaly detection has been explored by several researchers. Bertino and Islam (2017) discuss the use of machine learning in insider threat detection, noting that temporal patterns - specifically the times at which users access systems - are among the most reliable features for distinguishing normal from anomalous behavior. This finding directly motivated the use of scan hour as the primary feature in the Isolation Forest model implemented in this project.

Rule-based anomaly detection complements statistical methods by encoding domain knowledge explicitly [15]. Patcha and Park (2007) survey network intrusion detection systems and distinguish between signature-based methods, which detect known attack patterns, and anomaly-based methods, which identify deviations from established baselines. The hybrid approach used in this project running rule-based checks first, then applying the statistical model - reflects this distinction and ensures that both known suspicious patterns and novel behavioral deviations are captured.

2.4 Commercial and Existing Access Control Systems

In the industrial, commercial and institutional setting, modern access control systems are increasingly prevalent. Vendor-supplied commercial solutions like those offered by Honeywell, HID Global or Bosch, generally combine RFID or smart card authentication with a centralized backend system to manage identities, record logs and generate reports [10]. These systems frequently include extra functionalities like biometric authentication, cloud-based surveillance, mobile credential access and integration with enterprise security infrastructure.

While these platforms offer reliability and scalability, their proprietary hardware requirements, cloud dependency, and high deployment costs place them beyond the reach of small organizations, academic laboratories, and facilities with limited budgets [12]. Furthermore, closed commercial ecosystems offer limited flexibility for custom behavioral analytics or integration with experimental data processing pipelines. This gap motivates the development of modular, low-cost access control systems built on commodity hardware and open-source software systems that can deliver core access monitoring and behavioral intelligence without enterprise infrastructure. This project directly responds to that need.

3. System Design and Implementation

3.1 System Overview

The overall system architecture of the proposed access control system is shown in Fig. 1. The system follows an end-to-end pipeline starting from RFID-based user identification at the edge, followed by local decision making on the Arduino, and transmission of structured data to a backend system for storage,

analysis, and visualization. The architecture highlights the interaction between hardware components and software layers in achieving real-time monitoring and access control.

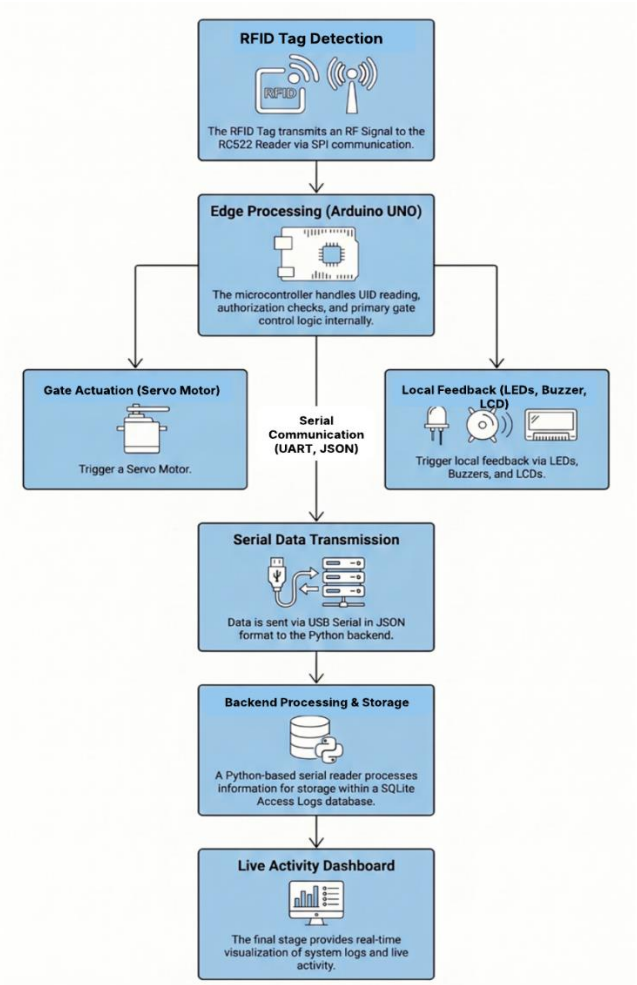

Fig. 1. System Architecture of the IoT-Based RFID Access Control System

3.2 Hardware Selection and Rationale

The Arduino UNO R3 was selected as the microcontroller platform for several reasons. It is widely supported, has an extensive library ecosystem, operates at 5V logic with a 3.3V regulated output pin for the RC522, and has sufficient digital I/O for all required peripherals. The ATmega328P processor running at 16 MHz provides adequate performance for the real-time scan processing required, and the 32KB flash with 2KB SRAM is sufficient when the code is written with memory efficiency in mind. The Grove Base Shield was used to simplify wiring for the Grove-compatible modules. Grove is an open-source hardware system developed by Seeed Studio that standardizes connector types for common sensor and actuator modules, reducing wiring errors and improving repeatability. The shield plugs directly onto the Arduino and exposes labeled Grove ports for digital, analog, and I2C connections. The RC522 RFID module was connected via jumper wires rather than a Grove connector because RC522 does not exist in the Grove ecosystem. The SPI pins on the Arduino are accessible through the base shield's pass-through headers. The module was powered from the 3.3V pin on the Arduino - this is a critical hardware constraint because the RC522 operates at 3.3V and connecting it to 5V will permanently damage the chip.

The Grove LCD RGB Backlight v4.0 was selected for its I2C interface, which requires only two signal lines regardless of the amount of data transmitted, and its ability to display colored backlight that was used to visually reinforce the access decision - green for granted, red for denied, blue for idle. The Grove Analog Servo was chosen for gate actuation. The servo operates in the 0 degree to 180 degree range, with 0 degrees representing the closed position and 90 degrees representing the open position. This range was sufficient to simulate a physical barrier being raised.

3.3 Pin Mapping and Conflict Avoidance

One of the first engineering challenges was assigning pins to all peripherals without conflicts. The SPI bus occupies D11, D12, and D13 permanently. The chip select for the RC522 was assigned to D10 and the reset line to D7. These six pins are exclusively reserved for the RFID module. The remaining peripherals were assigned to non-SPI pins: servo on D3 (a PWM-capable pin required for servo control), green LED on D4, red LED on D5, blue LED on D6, buzzer on D8, and the LCD on the I2C pins A4 and A5. This mapping ensures no two peripherals share a pin and the SPI bus is not interfered with by any other module.

3.4 Physical Prototype and Setup

A simple and practical set up was used to construct the physical prototype of the system to show how the access control system works. An Arduino UNO was attached to a Grove Base Shield that facilitated easier connection of most of the components like LEDs, buzzer, LCD display, and the servo motor using standard Grove connectors.

As the RC522 RFID module is not a Grove-compatible part, it was wired as a separate component, directly to the SPI pins on the Arduino, over jumper wires. Caution was observed when wiring this module since it works with 3.3V and linking it to 5V may destroy the device. The gate mechanism was modeled using the servo motor. A physical gate was not used but instead, the rotation of the servo between 0 degrees and 90 degrees was used to simulate the closing and opening of the gate. This gave a visual indication of either an access or a denial.

The components were all set up on a simple breadboard-like system without a decent enclosure. This prototype was aimed at concentrating on the functionality and integration of the system and not on the physical design. The arrangement however indicates how the system can be further extended to a real world system with appropriate casing and mounting. The Arduino was connected to the computer via USB where it supplied the serial communication with the Python backend. The entire system was stable and enabled testing of the entire system under varying conditions.

3.5 Arduino Firmware Design

- Arduino firmware is responsible for reading RFID cards, checking access, controlling the gate, and sending data to the backend system. Serial communication was set to a baud rate of 9600 to ensure stable and reliable communication between the Arduino and the Python backend.
- The system sends scan data in JSON format using a simple marker-based protocol:
  <{"uid":"B3 2B 1C 56","status":"allowed","event":"IN"}>
  The start (<) and end (>) markers help the backend identify complete messages and ignore incomplete or corrupted data.
- Since the Arduino UNO has limited memory (2KB SRAM), memory was handled carefully. Fixed-size character arrays were used instead of dynamic String objects to avoid memory fragmentation and ensure stable operation over time.

- Each RFID card UID is read as raw bytes and converted into a hexadecimal string format. This allows consistent and reliable comparison with stored authorized IDs. To track user presence, a simple mechanism was used with parallel arrays.
- Each user is assigned an IN or OUT state, which toggles every time the card is scanned. This helps the system determine whether a user is currently inside or outside.
- During testing, it was observed that the RC522 module required reinitialization in every loop cycle for consistent detection. A delay of 1 second was also added after each scan to prevent repeated readings when the card is kept near the reader.

3.6 Python Backend Architecture

The backend is organized according to clean architecture principles with strict separation of concerns across four layers.

- The backend is designed in a modular way by separating responsibilities into different layers. This helps in keeping the system organized and easy to maintain.
- The configuration layer stores all system settings such as serial port, database path, and anomaly detection thresholds in a single place using a settings class. This allows easy updates without changing the main code.
- The data layer handles database operations using SQLAlchemy. All interactions with the database are done through a repository class, which keeps the database logic separate from the rest of the system.
- The service layer is responsible for processing each scan event. It validates incoming data, stores it in the database, and runs anomaly detection. This layer contains the main application logic.
- The API layer is built using FastAPI and provides endpoints to access logs and dashboard data. These routes are simple and only return processed data from the backend.
- A separate serial communication module continuously reads data from the Arduino. It processes incoming messages and sends them directly to the backend service layer. This runs independently, so it does not affect the API performance.

3.7 Anomaly Detection Pipeline

The Anomaly Service implements a two-layer detection strategy.

Layer 1 - Rule-Based Checks:
The off-hours check compares the scan timestamp hour against configurable start and end hours defined in config.py. For this deployment, the window was set to 10AM through 3PM, reflecting a typical operational shift. Any scan outside this window is immediately flagged. The rapid-scan check queries the database for all scans of the same UID within the past 60 seconds. If five or more scans are found, the current scan is flagged. This implements a sliding window algorithm at the application layer - the window boundary is computed as the current time minus 60 seconds, and the query filters on both UID and timestamp. The consecutive denied streak check retrieves the five most recent logs for a given UID and checks whether the last three are all denials. Three or more consecutive denials indicate a sustained unauthorized access attempt.

Layer 2 - Isolation Forest:
The ML layer activates only when a user has at least 10 historical scan records - the minimum needed to train a meaningful model. The feature set is the hour of day extracted from each historical timestamp. The model is retrained fresh on each scan using all available history except the current record. The Isolation Forest is configured with a contamination parameter of 0.1, meaning approximately 10% of the training data is expected to be anomalous. The current scan's hour is then passed to the predict method. A return

value of -1 indicates an anomaly. When flagged, the reason string includes the hour that was flagged and a description of the user's expected pattern.

3.8 Dashboard Implementation

The dashboard is a single-page HTML application served as a static file from the FastAPI backend (Fig. 2). JavaScript fetch calls poll the API endpoints every 5 seconds and update the DOM in place without a full page reload.

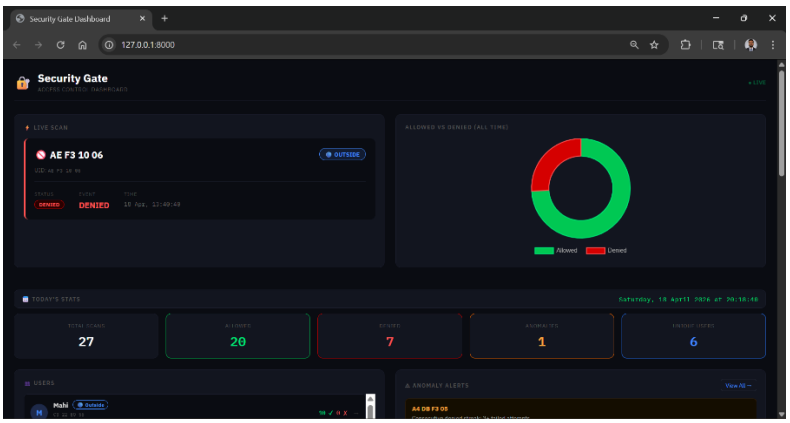

Fig. 2. Main Dashboard showing real-time access activity and statistics

The live scan panel shows the most recently processed scan event including the user's display name (resolved from a server-side user map), UID, status badge, event tag, timestamp, and presence badge indicating whether the user is currently inside or outside. Presence is determined by the most recent allowed event for that user - IN means inside, OUT means outside.

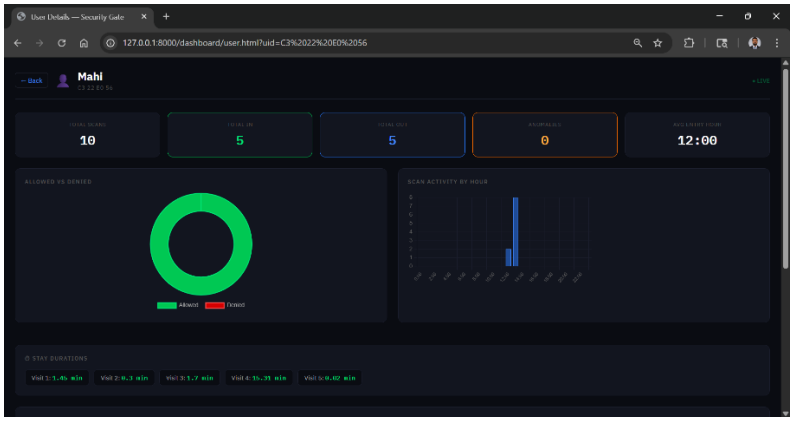

Fig. 3. User Detail Page showing individual user behavior and analytics

Detailed user-level analytics including scan patterns and stay duration are shown in Fig. 3. The today's stats section queries the /api/stats/today endpoint which filters the database to records with timestamps between local midnight and the current time. This count resets to zero automatically at midnight each day without any scheduled job because the query boundary is always recomputed dynamically from the current date. Detected anomalies are displayed in a dedicated alerts view as shown in Fig. 4.

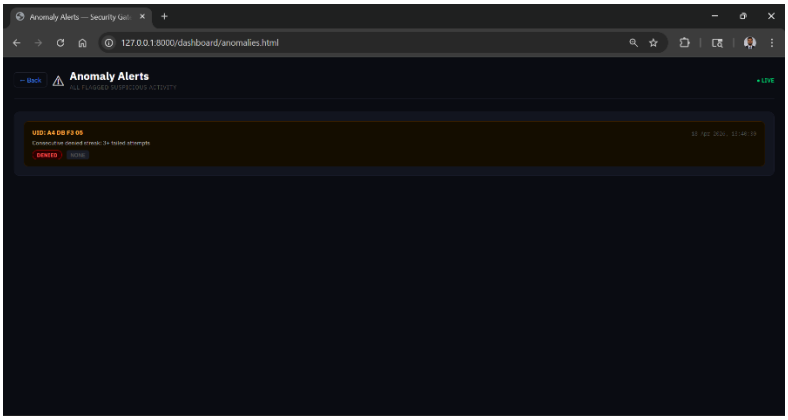

Fig. 4. Anomaly Alerts Page showing detected suspicious activities

The log filter accepts a date range and optional time range. When applied, a filter lock flag prevents the 5-second auto-refresh from overwriting the filtered results. The Reset button clears the filter and restores live view.

3.9 Testing and Validation

The system was tried with a combination of several RFID cards, both approved and unapproved. Verification was done to verify that authorized cards were swiped to unlock the gate and update the database and show changes on the dashboard in real time. Testing of the unauthorised cards was done to ensure that the access was denied and appropriate alerts were received.

Anomaly detection was checked by testing in different situations. Repeated scanning of the same card in a short period evoked the rapid scan detection rule. Several denied transactions with an unapproved card activated successfully the consecutive denial rule. Access when out of hours was also checked by simulating some scans which did not belong to the set time window and they were properly considered as anomalies.

Time taken by the system to scan a card and update the dashboard was found to be approximately 1-2 seconds. This involves processing on the Arduino, serial communication, database update and dashboard refresh. During testing, around 30 - 50 scans were done to make sure that the system is reliable when used continuously. On the whole, the system worked as anticipated and could support access control in real-time, logging, and simple behavior analysis without significant problems.

4. Challenges

- **RC522 Detection Reliability:** A key hardware problem was the ability to read the cards with the RC522 module. The first contacts with the Grove Base Shield led to inaccurate readings, which means that SPI communication is not the most reliable. The problem was resolved by testing various wiring configurations and examining the power supply after determining that the problem was with unstable connections and sensitivity of the reset pin. The reader of RFID was re-initialized with each loop cycle to enhance reliability. This is not a typical method; however, it gave consistent readings when tested.
- **Serial Port Conflicts:** In development, there was conflict between the Arduino IDE and Python backend attempting to share the same serial port. The problem was addressed by shutting the

Serial Monitor before starting the backend and stopping it before uploading code as only one program can use the port at a time.

- **Hardware Limitations:** The RC522 did not come with an appropriate connector and thus was required to be manually connected with jumper wires. This rendered the wiring less stable and had to be handled carefully. The other problem was that the power supply of the Arduino since other components like the servo motor, LEDs, LCD, and buzzer were attached. This would occasionally make the RFID module act inconsistently or fail to work. When testing, the system needed to be re-started by reconnecting the Arduino and re-uploading the code. Although this was successful with the prototype, a more reliable external power source would be needed in the real world.
- **Model Scalability:** The anomaly detection model is re-trained on each scan with accessible data. This is effective with small data sets but might not be effective with large data sets in the long term. In practical applications, batch training or incremental learning would be more applicable.

5. System Limitations

The system has a number of limitations that can be considered for future improvements. The Arduino firmware is hardcoded with the list of authorized users, so to add or remove authorized users, you have to upload new firmware to the device. The serial connection is USB based, which restricts the length of the cable and the Arduino must be physically close to the laptop with the backend running. The Isolation Forest model takes advantage of a single feature, thus not being sensitive to anomalies that arise in other dimensions than time of day. Web dashboard is not authenticated and anyone on the same local network can access the entire access log. The cloning of cards is not taken care of at the hardware level.

6. Conclusion

This project demonstrates that an effective IoT-based access security system can be built using low-cost and widely available hardware. By combining RFID authentication, servo-based gate control, serial communication, a Python backend, and a real-time dashboard, the system provides a complete solution for access control and monitoring. The addition of rule-based and machine learning-based anomaly detection further enhances the system by enabling basic behavioral analysis.

One of the key learnings from this project was the importance of building and testing each system component independently before full integration. This approach helped in identifying issues more clearly and reduced debugging complexity. Another important learning was the need for careful memory management in embedded systems, where design decisions directly impact system stability. There are several areas for improvement. The system could be improved to be no longer reliant on USB communication by using WiFi-enabled microcontrollers in the future. The list of authorized users can be transferred to the back-end database to enable dynamic updates. Also, the anomaly detection model can be improved by adding more features to enhance its accuracy and usefulness. In general, the system can be applicable to small and medium-size settings including academic labs, shared facilities, and small warehouses that need the simplest access control and surveillance without the expense of enterprise solutions.

References

1. Ganeshan, Chinmay, and Sanjay kumar Singh. “Smart Industrial System for Monitoring, Control and Security Using Internet of Things.” 2018 2nd International Conference on Trends in Electronics and Informatics (ICOEI), May 2018, 958–63. https://doi.org/10.1109/ICOEI.2018.8553835.
2. Chen, Min, Jia Liu, Shigang Chen, Yan Qiao, and Yuanqing Zheng. “DBF: A General Framework for Anomaly Detection in RFID Systems.” IEEE INFOCOM 2017 - IEEE Conference on Computer Communications, May 2017, 1–9. https://doi.org/10.1109/INFOCOM.2017.8056986.

3. Liang, Fei, Siye Wang, Ziwen Cao, Yue Feng, Shang Jiang, and Yanfang Zhang. “BPCluster: An Anomaly Detection Algorithm for RFID Trajectory Based on Probability.” 2023 IEEE Symposium on Computers and Communications (ISCC), July 2023, 844–50. https://doi.org/10.1109/ISCC58397.2023.10218107.
4. Chunli, Liu, and Huang Linna. “A Study and Realization of RFID Security Authentication System.” 2012 International Conference on Computer Science and Information Processing (CSIP), August 2012, 1382–85. https://doi.org/10.1109/CSIP.2012.6309120.
5. Geetha, V., and Bavya Balakrishnan. “A User Authentication and Access Control Scheme for IoT-Based Healthcare Using Blockchain.” 2021 12th International Conference on Computing Communication and Networking Technologies (ICCCNT), July 2021, 01–07. https://doi.org/10.1109/ICCCNT51525.2021.9579992.
6. Ganapathy, Jayanthi, Nachiappan Nachiappan, Sreevijnya Menakur, and Purushothaman Ramachandran. “iTRAC: Intelligent Tracking and Real-Time Analysis on Cloud Using RFID and MQTT.” TENCON 2025 - 2025 IEEE Region 10 Conference (TENCON), October 2025, 1683–87. https://doi.org/10.1109/TENCON66050.2025.11374930.
7. Humayun, Mamoona, Noshina Tariq, Majed Alfayad, Muhammad Zakwan, Ghadah Alwakid, and Mohammed Assiri. “Securing the Internet of Things in Artificial Intelligence Era: A Comprehensive Survey.” IEEE Access 12 (2024): 25469–90. https://doi.org/10.1109/ACCESS.2024.3365634.
8. S, Joshika, A. Senthil Kumar, L. Santhi, and R. Sakthivel. “A Smart Security System Using IOT.” 2024 International Conference on Intelligent Computing and Emerging Communication Technologies (ICEC), November 2024, 1–4. https://doi.org/10.1109/ICEC59683.2024.10837108.
9. Escobal, Shem Japhet T., Mariciel M. Teogangco, Omar B. Jordan, Wilfredo N. Leaño, and Michael O. De La Peña. “Development of a Software Dashboard for A Locally-Made IoT Gateway Device.” 2025 International Conference on Communication, Computing, Networking, and Control in Cyber-Physical Systems (CCNCPS), June 2025, 111–17. https://doi.org/10.1109/CCNCPS66785.2025.11135719.
10. Nair, Adwaith A., R. Adithyan, Akshara Unni, and Shankar Nalinakshan. “RFID Door Lock Access Control Systems: Trends, Technologies and Applications.” 2025 3rd International Conference on Intelligent Data Communication Technologies and Internet of Things (IDCIoT), February 2025, 906–12. https://doi.org/10.1109/IDCIOT64235.2025.10914928.
11. Vikram, D., N. Jeyakkannan, SP Santhoshkumar, R. Karthika, P. Punitha, and H. Ajina. “Securing IoT: Anomaly Detection and Behavioral Analysis for Data Integrity.” 2024 5th International Conference on Intelligent Communication Technologies and Virtual Mobile Networks (ICICV), March 2024, 892–96. https://doi.org/10.1109/ICICV62344.2024.00147.
12. Allah, Osman Abd, Suliman Abdalla, Marwa Mekki, and Alaa Awadallah. “RFID Based Access Control and Registration System.” 2018 International Conference on Computer, Control, Electrical, and Electronics Engineering (ICCCEEE), August 2018, 1–4. https://doi.org/10.1109/ICCCEEE.2018.8515794.
13. Masvisvi, Carol, Ravi Samikannu, R. Rohini, G. Kanimozhi, R. Kavitha, and K. Mohana Sundaram. “RFID-Enabled Access Control with Integrated User Database Logging.” 2025 10th International Conference on Communication and Electronics Systems (ICCES), October 2025, 159–64. https://doi.org/10.1109/ICCES67310.2025.11336906.
14. Kareem, Mohammed Shaker, and Lamia AbedNoor Muhammed. “Anomaly Detection in Streaming Data Using Isolation Forest.” 2024 Seventh International Women in Data Science Conference at Prince Sultan University (WiDS PSU), March 2024, 223–28. https://doi.org/10.1109/WiDS-PSU61003.2024.00052.
15. Costa, Gianni, Agostino Forestiero, and Riccardo Ortale. “Rule-Based Detection of Anomalous Patterns in Device Behavior for Explainable IoT Security.” IEEE Transactions on Services Computing 16, no. 6 (2023): 4514–25. https://doi.org/10.1109/TSC.2023.3327822.